LOCUS: A Cue-Policy Interface for Storage-Separated Threshold Private-Key Recovery

Anonymous Author(s)

Abstract

Long-term private-key recovery needs a human-facing input that can be supplied after device loss without turning a stored backup into an offline guessing target. LOCUS introduces a versioned cue-policy boundary that converts deployment-chosen structured input into canonical client-local bytes for threshold password-authenticated secret sharing (TPASS). The resulting password mediates recovery of an independently sampled group secret, from which the client derives the key that encrypts the private-key backup. The cloud stores the ciphertext, while recovery parties store threshold state and backup bindings; under the TPASS assumptions, cloud compromise, below-threshold party compromise, or their combination does not create an offline cue-testing oracle. A hybrid Rust/Python prototype demonstrates the interface through a reference policy requiring three location–person pairs, a native Rust/Ristretto255 TPASS core, AES-256-GCM backup protection, separated S3-compatible storage, and authenticated same-host party services. Conformance tests cover canonical ordering, ambiguity, malformed input, and resolver drift. Across 30 measurements per operation, median enrollment and successful recovery took 250 and 442 ms, respectively; recovery with one TPASS party unavailable took 411 ms. Three isolated synthetic snapshot experiments—cloud-only, one party in the deployed 2-of-3 profile, and their combination—reported no local candidate signal for either of two fixed candidates under networkless, read-only boundaries. These results demonstrate the implemented cue-policy, native-recovery, and storage-separation boundaries for the evaluated same-host profile.

CCS Concepts

• **Security and privacy → Key management**; **Security protocols**; *Usability in security and privacy*.

Keywords

private-key recovery, offline guessing resistance, threshold password-authenticated secret sharing, decentralized recovery, usable security, cloud backup, account recovery

ACM Reference Format:

Anonymous Author(s). 2027. LOCUS: A Cue-Policy Interface for Storage-Separated Threshold Private-Key Recovery. In *Proceedings of the 22nd ACM Asia Conference on Computer and Communications Security (ASIACCS '27), July 12–16, 2027, Macau, China.* ACM, New York, NY, USA, **??** pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

ASIACCS '27, Macau, China

ACM ISBN 979-8-XXXX-XXXX-X/27/07
https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 Introduction

Private keys protect encrypted data, self-custodied assets, identities, and account credentials, yet loss of the only usable copy can make recovery impossible. Existing mechanisms trade availability against security: custodial recovery concentrates authority, social recovery delegates decisions to guardians, conventional password-protected backups can enable offline guessing after leakage, and high-entropy bearer strings must remain available for years. LOCUS seeks private-key recovery after device loss without giving one service unilateral recovery power or placing an offline cue-testing verifier in stored state.

Prior studies of graphical cues, geographic passwords, and location-based fallback authentication show delayed reproduction in particular evaluated schemes [**? ? ? ?**]. They motivate structured cues as an interface class, not the usability of a LOCUS policy. The systems gap is reproducibility: TPASS consumes stable password bytes, while human-derived information may be ambiguous, provider-dependent, or publicly predictable [**? ?**]. A deployment therefore needs an explicit contract for accepted input, resolution, normalization, ordering, ambiguity, drift, versioning, and failure.

LOCUS supplies that contract as a client-side cue policy. The policy maps structured input to canonical bytes or fails; public metadata identifies the rules without revealing selected cues or a testing digest. The client binds the output to a recovery identity and uses it as the password input to the TPASS construction of Yi et al. [**? ?**]. TPASS recovers an independently sampled group secret, from which the client derives the backup-wrapping key. The cloud stores the ciphertext, while recovery parties store threshold state and backup bindings. Under the TPASS assumptions, the cloud, fewer than t parties, or their combination cannot test cue guesses offline; guesses remain online protocol events.

The prototype instantiates the policy interface with a three location–person-pair reference policy and integrates a native Ristretto255 TPASS core, standard backup cryptography, S3-compatible storage, and authenticated same-host party services. Its evaluation covers deterministic policy behavior, native recovery cost, and isolated cloud, one-party, and combined persistent snapshots. A bounded state exploration also demonstrates why the prototype's quorum ledger does not establish rollback-resistant global attempt control.

This paper contributes: (1) a versioned cue-policy interface for reproducible client-local TPASS input; (2) a storage-separated recovery architecture in which TPASS recovers random keying material rather than using cues directly for encryption; (3) a native prototype and evaluation of one reference policy, the end-to-end recovery path, and persistent state boundaries; and (4) a claim-scoped analysis of offline and online guessing, resolver leakage, and the partial ledger's rollback counterexample.

2 Problem Setting and Threat Model

2.1 Problem Setting

Let U denote a user who holds a private key sk_U. The goal of LOCUS is to let U recover sk_U after losing the original device, using structured input admitted by a deployment-defined cue policy, while avoiding a single recovery provider that can reconstruct the key and avoiding stored material that enables offline cue guessing.

LOCUS is intended for infrequent, high-value recovery events rather than daily authentication. A deployment chooses a human-facing cue scheme and records a public version identifier for the rules that convert selected input into canonical bytes. The security analysis treats the resulting input as potentially predictable; policy-specific memorability, reproduction, and selection behavior require separate human evaluation.

2.2 Entities

LOCUS involves three entity types.

User. The user initially holds a private key sk_U. During enrollment, the user selects structured recovery information under the deployment's cue policy. The client canonicalizes that selection, derives the TPASS password input, encrypts sk_U, and distributes recovery state. During recovery, the user operates from a fresh device and supplies candidate input under the same public policy version.

Cloud service. The cloud service stores the encrypted backup object. It is not trusted for confidentiality or integrity. Integrity failures are detected by authenticated encryption and by comparing backup-object digests stored by recovery parties. The cloud is trusted only for availability, or the backup object may be replicated across multiple storage providers.

Recovery parties. Recovery parties store threshold recovery state and participate online during recovery. A threshold parameter t determines how many parties are required. The prototype records request-bound local accounting before a secret-dependent response, but the bounded rollback result shows that its signed durable ledger does not establish a global attempt property.

2.3 Threat Model

We consider adversaries that may compromise the cloud service, compromise fewer than t recovery parties, observe recovery traffic over an untrusted relay, observe or control a resolver provider, or attempt online guessing through the recovery interface. Enrollment channels from the user to recovery parties are assumed to be authenticated and confidential because they carry recovery-party state. Because a cue policy may admit personal information, the model explicitly includes adversaries who infer candidate inputs from public traces or social knowledge.

We distinguish the following adversary classes.

Cloud adversary. The adversary obtains the cloud backup object, including the encrypted private key and public policy metadata. The adversary may delete, replay, or substitute backup objects.

Below-threshold party adversary. The adversary compromises fewer than t recovery parties and obtains their stored state. The adversary may also know public recovery metadata and may collude with the cloud adversary.

Online guessing adversary. The adversary attempts recovery sessions using candidate inputs admitted by the active cue policy. Candidates may be generated from public traces, social knowledge, resolver leakage, or ordinary password-guessing strategies. Such attempts require interaction with recovery parties. The prototype instruments local party-side attempt state, but any adversarially enforced rate limit is a deployment assumption.

Public-information adversary. The adversary uses public information about the user, such as public posts, photographs, workplaces, travel history, family information, or social-media links, to predict likely personal cues.

Social-knowledge adversary. The adversary knows the user personally and may know relatives, friends, schools, workplaces, frequent locations, or life events that the user might select as recovery cues.

Provider-observation adversary. The adversary observes queries and selected candidates submitted to an external resolver. Local hashing does not hide these queries because resolution occurs before hashing.

Table **??** summarizes the main attacker goals considered in the analysis and the corresponding LOCUS security response.

3 LOCUS Requirements

This section states the requirements that the construction must satisfy. Section **??** then gives the cue processing and cryptographic protocol that realize them.

3.1 Design Requirements

LOCUS is designed around the following requirements.

R1: No offline cue-testing material. Recovery parties must not store password hashes, cue-derived identifiers, answer records, or any other verifier that lets a compromised party test guessed recovery cues without participating in an online threshold recovery session.

R2: Storage-recovery separation. The encrypted private key and the threshold recovery state must be stored by different components.

R3: Threshold recovery. A user who has the encrypted backup object, supplies the correct cues, and obtains participation from at least t valid recovery parties should recover sk_U.

R4: Online guessing boundary. Incorrect cue guesses should require online recovery sessions. A deployment that needs a bounded number of evaluations must independently provide rate limiting, admission, cooldowns, audit handling, and false-lockout recovery.

R5: Explicit cue-policy processing. Each supported policy must identify its accepted input structure, resolver behavior, validation and normalization rules, canonical ordering, ambiguity handling, and version. Applying the same policy to equivalent admitted selections must produce the same canonical protocol input; unsupported, ambiguous, or drifting input must fail explicitly rather than trigger alternative guesses.

R6: Limited stored leakage. The cloud and recovery parties should not store raw cues, selected resolver-record identifiers, per-cue identifiers, the derived password, the recovered group secret, or the wrapping key.

Table 1: Attacker goals and intended LOCUS response.

Adversary	Goal	LOCUS response	Residual risk
Cloud-only compromise	Decrypt C_U or test guessed cues offline from B_U	Cloud stores ciphertext and public metadata, but no TPASS states, raw cues, cue identifiers, p_M, S_R, wrapping key, or verifier	Deletion, rollback, and substitution affect availability or require digest checks
Fewer than t recovery parties	Reconstruct S_R, test cue guesses locally, or recover sk_U	TPASS threshold security prevents below-threshold reconstruction and party records contain no encrypted key or password verifier	Online guessing remains possible whenever parties participate; complete attempt enforcement is unfinished
Cloud plus fewer than t parties	Combine ciphertext with partial recovery state to test cues offline	Storage-recovery separation and TPASS prevent a local cue-testing oracle below threshold	Same online-guessing and availability risks as above
At least t compromised parties	Recover threshold secret or assist malicious recovery	Outside the threshold-compromise assumption	Must be addressed by party selection, monitoring, replacement, and operational controls
Online guessing client	Submit candidate cues through recovery sessions	Attempts require online party participation; the design requires counting before response	The complete distributed bound is assumed, not established by the current prototype
Public-information attacker	Infer likely cue values from public traces	No offline oracle; guesses must be tested online through TPASS	Obvious cues may be guessed in few attempts
Social-knowledge attacker	Use personal knowledge to predict recovery inputs	Same online-only testing boundary; no claim that social knowledge is hidden	Strong social knowledge can reduce cue uncertainty substantially
Resolver observer	Observe lookup queries or selected candidates	Resolver privacy is not provided by local hashing after lookup	Use local, self-hosted, cached, or private lookup mechanisms when resolver privacy is required
Malicious cloud or stale backup	Substitute, roll back, corrupt, or delete backup object	Party-stored digest and AEAD detect substitution or corruption during recovery	Deletion and unavailable current backups remain availability failures

R7: Explicit privacy boundary. When a cue policy uses an external resolver, the design must distinguish storage privacy from resolver privacy. Hashing after lookup prevents leakage to recovery parties and the cloud, but not to the resolver itself.

4 Protocol Construction

Before formalizing notation and cue processing, consider a typical recovery. During enrollment, the user chooses structured cues under a deployment-defined cue policy. The client applies that policy locally—including resolution when the policy requires it—derives the TPASS password, encrypts sk_U, uploads the backup object to the cloud, and sends each recovery party only its TPASS state and public recovery metadata. After device loss, a fresh client downloads the backup, accepts candidate cues, applies the enrolled public policy version, and asks a threshold set of parties to run TPASS. If the candidate cues reproduce the enrolled canonical policy output, the client obtains the same group secret and decrypts sk_U; otherwise recovery fails through the online protocol and is counted by the participating parties.

4.1 Notation

Let U denote the user and let sk_U denote the protected private key. Let

$$\{P_1, \ldots, P_n\}$$

be the recovery parties, and let t be the recovery threshold.

LOCUS uses the TPASS protocol of Yi et al. [**?** **?**] for this layer; the notation and equations below follow that construction. Let $\mathbb{G}$ be a cyclic group of prime order q, with generators $g_1, g_2 \in \mathbb{G}$, where no party knows the discrete logarithm of g_2 with respect to g_1, as required by that construction.

The protocol uses deterministic encoding $\mathsf{Encode}(\cdot)$, a binary hash

$$H_b : \{0,1\}^* \to \{0,1\}^\lambda,$$

a scalar hash

$$H_q : \{0,1\}^* \to \mathbb{Z}_q,$$

HKDF [**?**], and an authenticated-encryption-with-associated-data scheme AEAD [**?**].

4.2 Cue-Policy Boundary

4.2.1 Policy Contract. Let M be the structured recovery input selected by a user and let v_M identify a deployment-supported cue policy. The policy is a client-side transformation

$$\mathsf{CuePolicy}_{v_M}(M) \to Z_M \text{ or } \perp,$$

where Z_M is a canonical byte string. Each policy specifies its accepted input types and cardinality, resolver profile, validation and normalization rules, ordering and duplicate behavior, ambiguity and drift handling, and canonical encoding. The policy identifier is included in Z_M, so changing a rule requires a new identifier and recovery epoch rather than reinterpretation of existing state.

The policy may use a resolver to turn human-facing descriptions into stable records, but it never searches alternatives through TPASS. Unsupported, ambiguous, missing, or drifting input returns $\perp$. Raw queries, resolver records, intermediate descriptors, Z_M, and the derived password remain client-local. Public context metadata Π_{ctx} identifies the policy rules needed by a fresh client but contains no selected cue, user-specific record identifier, or cue-testing digest.

4.2.2 Reference Policy. The prototype instantiates this contract with

$$v_M = \texttt{LOCUS-location-person-set-v1},$$
$$v_\mu = \texttt{LOCUS-location-person-pair-v1}.$$

For this policy,

$$M = \{\mu_1, \mu_2, \mu_3\}$$

contains exactly three location–person pairs. Each pair contains a WGS 84 point and one email or E.164 contact channel. Coordinates are parsed from exact decimal strings, multiplied by 10^4, and rounded half-to-even; email values use a constrained lowercase ASCII form, and phone values use canonical E.164. Names, relationship labels, provider record identifiers, and unselected contact channels are display-only.

For each validated pair μ_j, the client encodes

$$d_j = \mathsf{Encode}(\mathsf{location}_j, \mathsf{person}_j, v_\mu).$$

The three full pair objects are ordered by

$$\big(H_b(\texttt{"LOCUS/cue-pair/v1"}, d_j), d_j\big),$$

using the encoding as a collision tie-breaker. Duplicate canonical locations, people, or complete pairs are rejected. The complete canonical recovery input is

$$Z_M = \mathsf{Encode}\big(\{\texttt{"pairs"} : [d_{(1)}, d_{(2)}, d_{(3)}], \texttt{"version"} : v_M\}\big).$$

The pinned synthetic fixture produces 511 bytes. This reference policy demonstrates the contract with nontrivial resolution, normalization, ordering, and drift behavior; it is not a restriction on the cue types or cardinalities that another deployment may define.

4.3 Cue-Derived Recovery Password

Let bid be a fresh 16-byte public backup identifier and define the reference epoch-1 recovery identity

$$ID_R = \texttt{"LOCUS-compose-recovery-v1:"} \,\|\, \mathsf{bid}.$$

LOCUS derives a scalar recovery password from the output of the active policy:

$$p_M \in \mathbb{Z}_q$$

as

$$p_M = H_q^{\mathsf{TPASS}}\big(\texttt{"password"}, ID_R, Z_M\big). \tag{1}$$

Here H_q^{TPASS} is SHA-512 reduced to a Ristretto scalar after length-prefixing the protocol domain `LOCUS-TPASS-YI-ZK-RISTRETTO255-v1`, operation label, recovery identity, and policy output. The public recovery nonce used below is not a password input. The value p_M is not an encryption key and is not stored by the cloud service or by any recovery party.

4.4 TPASS Interface Used by LOCUS

A threshold password-authenticated secret-sharing protocol allows a client to recover a secret from any threshold subset of servers using a password, while coalitions below the threshold cannot recover the secret or test password guesses offline. LOCUS instantiates its TPASS layer with the zero-knowledge-proof-based construction of Yi et al. [**? ?**]. It does not claim a new TPASS construction; its contribution is the private-key recovery architecture built around this instantiation: cue-derived password input, separated cloud backup storage, recovery-party state separation, and explicit online-risk boundaries. Password-protected secret sharing is related to earlier work on password-protected and password-authenticated recovery of secrets [**? ?**].

The main construction uses the following abstract interface:

$$\mathsf{TPASS.Setup}(ID_R, p_M, t, n) \to (\mathsf{pp}, \{\mathsf{st}_i\}_{i=1}^n, S_R),$$
$$\mathsf{TPASS.Recover}(ID_R, p'_M, I, \{\mathsf{st}_i\}_{i\in I}) \to S'_R \text{ or } \perp.$$

Here ID_R is the recovery identity, p_M is the cue-derived password, and I is a threshold set of participating TPASS parties. Setup internally samples the recovery exponent and secret-shares the password, exponent, and digest scalar.

The recovered 32-byte encoded TPASS group secret is

$$S_R = g_2^{z_R}.$$

LOCUS uses S_R to derive a symmetric wrapping key. The protected private key is encrypted under that wrapping key rather than being secret-shared directly.

4.5 Recovery Secret and Key Wrapping

During enrollment, TPASS samples

$$z_R \leftarrow \mathbb{Z}_q$$

and defines the TPASS-recovered group secret as

$$S_R = g_2^{z_R}.$$

The native core computes the digest exponent

$$\theta_R = H_q^{\mathsf{TPASS}}\big(\texttt{"secret-digest"}, ID_R, \mathsf{Encode}(S_R)\big). \tag{2}$$

Let e be the explicit positive backup epoch and let v_R be a fresh 16-byte recovery nonce. There is no intermediate recovery key. The wrapping key is derived directly as

$$K_{\mathsf{wrap}} = \mathsf{HKDF\text{-}SHA256}\big(\mathsf{IKM} = \mathsf{Encode}(S_R),\ \mathsf{salt} = v_R,\ \mathsf{info} = \mathsf{Encode}(\{\texttt{"purpose"} : \texttt{"LOCUS-wrap"}, \texttt{"bid"} : \mathsf{bid}, \texttt{"epoch"} : e\}),\ L = 32\big). \tag{3}$$

Define A_B as the canonical encoding of the associated-data format and backup-format versions, bid, e, v_R, complete TPASS public parameters, cue/context-policy metadata, security-policy metadata, and the sealed format/algorithm identifiers. With a separate fresh 12-byte GCM nonce, the private key is encrypted as

$$C_U = \mathsf{AES\text{-}256\text{-}GCM.Enc}_{K_{\mathsf{wrap}}}\big(sk_U; \mathsf{aad} = A_B\big). \tag{4}$$

4.6 Backup Object and Recovery-Party Records

Before adding its self-digest, the public encrypted backup is

$$\bar{B}_U = \big(\mathsf{v}_B, \mathsf{bid}, e, v_R, C_U, \mathsf{pp}_{\mathsf{YHCLZK}}, \Pi_{\mathsf{ctx}}, \Pi_{\mathsf{sec}}\big),$$

where v_B = `LOCUS-reference-backup-v4`; Π_{ctx} is authenticated public metadata identifying the active cue policy; and Π_{sec} records the local attempt budget and cooldown. Neither policy field contains a selected cue, resolver-record identifier, canonical descriptor, or local cue-testing verifier.

The backup-object digest is

$$h_B = H_b(\texttt{"LOCUS-backup-digest-v1"}, \mathsf{Encode}(\bar{B}_U)),$$

and $B_U = (\bar{B}_U, h_B)$. The canonical cloud envelope repeats (bid, e, h_B) and embeds B_U; readers require exact agreement. This digest binds recovery-party metadata to the cloud object and helps a recovering

Algorithm 1 LOCUS Backup Phase

Input: Private key sk_U; structured input M; supported cue policy v_M; native threshold parameters (t, n_T); TPASS parties $\{P_i\}_{i=1}^{n_T}$; authorizers $\{A_j\}_{j=1}^{n_A}$; cloud service C
Output: Cloud backup object B_U, TPASS records $\{R_i\}_{i=1}^{n_T}$, and initialized authorizer state

Apply $\mathsf{CuePolicy}_{v_M}(M)$ to obtain Z_M or fail
Sample fresh 16-byte bid; set $e \leftarrow 1$
Set $ID_R \leftarrow$ `"LOCUS-compose-recovery-v1:"`$\|$bid
Derive p_M using Eq. (??)
Run $\mathsf{TPASS.Setup}(ID_R, p_M, t, n_T)$ to obtain $(\mathsf{pp}_{\mathsf{YHCLZK}}, \{\mathsf{st}_i\}_{i=1}^{n_T}, S_R)$
Sample fresh 16-byte v_R; derive K_{wrap} using Eq. (??)
Form A_B and encrypt sk_U using Eq. (??)
Form $\bar{B}_U$, compute h_B, and set $B_U = (\bar{B}_U, h_B)$
Upload the canonical envelope and obtain cref = (bid, e, h_B)
for $i = 1$ to n_T **do**
    Form $R_i = (\mathsf{bid}, e, ID_R, \mathsf{st}_i, \mathsf{cref}, h_B, \Pi_{\mathsf{sec}})$
    Send R_i to P_i over an authenticated confidential channel
end for
for $j = 1$ to n_A **do**
    Initialize A_j's local attempt-control state for bid
end for
Erase $p_M, S_R, K_{\mathsf{wrap}}$, raw cues, resolver records, canonical descriptors, Z_M, and setup randomness
return B_U and $\{R_i\}_{i=1}^{n_T}$

Algorithm 2 LOCUS Recovery Phase

Input: Candidate structured input M'; enrolled cue policy v_M; exact cloud reference (bid, e, h_B); authenticated party endpoints
Output: Recovered private key sk_U or $\perp$

Download and validate the exact canonical envelope from C
Parse $B_U = (\bar{B}_U, h_B)$ and require matching (bid, e, h_B)
Validate the TPASS public parameters
Load ID_R and v_M; apply $\mathsf{CuePolicy}_{v_M}(M')$ to obtain $Z_{M'}$ or fail
Derive p'_M using Eq. (??) with $Z_{M'}$
Select one fixed healthy threshold set I from consistent party summaries
Durably authorize and count the request before native commitment generation
Run $\mathsf{TPASS.Recover}(ID_R, p'_M, I, \{\mathsf{st}_i\}_{i\in I})$
if TPASS recovery fails **then**
    return $\perp$
end if
Let S'_R be the recovered group secret
Derive K'_{wrap} using Eq. (??) with S'_R
Decrypt C_U using authenticated metadata A_B
if AEAD decryption fails **then**
    return $\perp$
else
    return sk_U
end if

client with current honest metadata detect cloud-object substitution, rollback, or cue-policy confusion before attempting decryption. It does not detect rollback of the authoritative party metadata itself. Tampering with C_U is also detected by AEAD decryption failure.

For each recovery party P_i, TPASS setup creates a server state

$$\mathsf{st}_i = (i, p_i, z_i, \theta_i),$$

where p_i, z_i, θ_i are TPASS shares corresponding to p_M, z_R, θ_R. These shares are produced using threshold secret sharing, following the standard Shamir setting [**?**].

Each party receives and stores the record

$$R_i = (\mathsf{bid}, e, ID_R, \mathsf{st}_i, \mathsf{cref}, h_B, \Pi_{\mathsf{sec}}),$$

where cref = (bid, e, h_B) is the exact cloud reference. In the evaluated deployment, TPASS is 2-of-3 and only parties 1–3 store st_i; five authenticated processes hold authorizer identity, configuration, durable attempt-ledger, phase, and audit state. Parties 4–5 are authorizers but hold no TPASS state. No party record contains the encrypted private key, raw cues, selected resolver-record identifiers, Z_M, recovered group secret, wrapping key, or password verifier.

4.7 Backup Phase

Algorithm **??** summarizes enrollment. The client applies the selected cue policy and derives p_M. Native TPASS setup samples the protected secret, returns S_R to the enrollment client, and distributes state across the TPASS parties. The client directly derives the wrapping key and encrypts the private key. The cloud stores the canonical envelope; each TPASS-capable party stores only its own state and bindings.

The setup channel to each TPASS party must be authenticated and confidential because R_i contains st_i. After enrollment, the client erases the cue-derived password, group secret, wrapping key, raw cues, resolver records, canonical descriptors, canonical recovery-input bytes, and setup randomness.

Table 2: State separation in LOCUS.

Component	Stored state
Cloud service C	$B_U = (\mathsf{v}_B, \mathsf{bid}, e, v_R, C_U, \mathsf{pp}_{\mathsf{YHCLZK}}, \Pi_{\mathsf{ctx}}, \Pi_{\mathsf{sec}}, h_B)$
TPASS-capable party P_i	$R_i = (\mathsf{bid}, e, ID_R, \mathsf{st}_i, \mathsf{cref}, h_B, \Pi_{\mathsf{sec}})$ and attempt-control state
Authorizer-only party	Identity, configuration, attempt-ledger, phase, and audit state; no st_i
Resolver provider	May observe raw lookup queries and selected candidates during resolution
User after setup	No LOCUS-maintained cue verifier or setup secret; recovery requires user-supplied input

4.8 Recovery Phase

Algorithm **??** summarizes recovery. The fresh client fetches the exact party-pinned cloud object before attempt consumption, validates its canonical envelope and digest, validates the TPASS public parameters, and rederives p'_M. If M' canonicalizes to the same bytes as M, then $p'_M = p_M$.

The client fixes one healthy TPASS subset, obtains a signed authorization certificate, and each selected party durably installs it before generating a secret-dependent commitment. The parties then execute TPASS recovery. If it succeeds, the client obtains $S'_R = S_R$, derives the wrapping key, and authenticates/decrypts C_U. Wrong input, unhandled policy drift, insufficient parties, an incorrect cloud object, or malformed responses returns $\perp$. The client does not reveal its final TPASS or AEAD outcome to the parties.

4.9 State Separation

Table **??** summarizes which component stores which state.

No single recovery party stores C_U, p_M, S_R, K_{wrap}, raw cues, selected resolver-record identifiers, Z_M, or a password verifier. The cloud service stores C_U and public policy metadata but not the threshold states needed to recover S_R.

5 Implementation

LOCUS is implemented as a hybrid Rust/Python research prototype. The prototype realizes the cue-policy contract with `LOCUS-location-person-set-v1`, whose exact three-pair canonicalizer feeds a Rust/Ristretto255 TPASS core through a narrow Python binding. The remaining path combines AES-256-GCM backup encryption, an S3-compatible store, mutually authenticated recovery-party services, and per-party durable SQLite state in a same-host container deployment. The client derives p_M, encrypts the protected key, and provisions one native TPASS state to each of parties 1–3. Five authenticated processes participate in signed authorization; parties 4–5 are authorizer-only. Recovery validates the exact cloud reference and policy version, rederives p'_M, invokes native TPASS with one fixed 2-of-3 subset, validates the digest relation, and opens the backup only if all checks succeed.

The reference policy is backed by a deterministic resolver fixture and pinned canonical/drift corpora covering coordinate rounding, ASCII email/E.164 normalization, all six pair orders, duplicate classes, ambiguity, missing records, display renames, reindexing, and provider-version drift. The `LOCUS-reference-backup-v4` format authenticates the reference-policy identifier; provisioning, layout audit, the client, and fetched cloud objects require that identifier before authorization. A different cue policy would require its own identifier, canonicalization contract, resolver behavior, test vectors, and recovery epoch. The software vectors establish deterministic behavior for the reference policy, not memorability, entropy, or alternate-client interoperability.

The artifact models three storage surfaces. The cloud object contains the backup identifier, explicit epoch, recovery nonce, encrypted private key, TPASS public parameters, context/security-policy metadata, and a complete-object digest. It contains no raw cues, Z_M, p_M, S_R, wrapping key, or party state. Each TPASS-capable party stores its own native state, exact cloud reference, expected digest, security policy, and local attempt-control state; authorizer-only parties store no TPASS state. Resolver providers are simulated by local records; live map, contact, and social-network APIs are outside the reproducible path.

The prototype has three TPASS backends. The default backend exposes a Rust/Ristretto255 implementation of the Yi et al. zero-knowledge construction through a narrow PyO3 boundary and stores canonical versioned public parameters and per-party state. A simulator and a fixed toy safe-prime backend remain available only through explicit test/development selection. The complete local flow uses deterministic JSON encoding, domain-separated hashing, HKDF-SHA-256, and AES-256-GCM through pinned libraries. The versioned ciphertext format uses a fresh 96-bit nonce and authenticates the backup identifier, recovery nonce, TPASS parameters, cue policy, security policy, and cipher-suite identifiers as associated data. These are research-grade implementation choices and have not received an independent cryptographic audit.

Failure handling is explicit because several security claims depend on where rejection occurs. Recovery fails for an unsupported policy or deployment profile, malformed TPASS public parameters or party state, backup-digest mismatch, stale or substituted cloud objects, insufficient parties, exhausted local counters, wrong cues, failed TPASS consistency, and ciphertext authentication failure. In the tested path, a signed certificate is durably installed before native commitment generation, so hiding the client's final outcome does not restore that local count. This does not establish a rollback-resistant global bound. Paper-facing measurements use the retained native 2-of-3 same-host profile; variable-input scaffolding, simulator runs, and toy-backend benchmarks are excluded from that evidence.

6 Evaluation

LOCUS is evaluated along four axes: reference cue-policy conformance, prototype cost, persistent state and adversarial failure boundaries, and model-based online guessing. The deployment experiments use provenance-bound, aggregate-only records from clean commit `12ca815` and one pseudonymously labeled same-host machine. The guessing analysis is conditional on an externally enforced attempt budget and assumed cue uncertainty. The evaluation contains no LOCUS human-subject experiment and therefore makes no comparative memorability claim for the reference policy.

6.1 Reference Cue-Policy Conformance

The reference-policy corpus binds one synthetic selection to a 511-byte canonical encoding. All six input orders produce those same bytes, while changing a location–person association, quantized location, selected channel, channel type, or channel value changes the encoding. Display-name and provider-record reindexing that preserve canonical values leave it unchanged. Unsupported versions, incorrect pair counts, duplicate locations or people, malformed coordinates or contact channels, unresolved ambiguity, missing records, and policy-relevant drift fail before an online attempt is requested.

These tests exercise the policy contract's determinism and fail-closed boundary for one concrete instantiation. They do not evaluate whether a person can reproduce the selected information, whether another cue policy has comparable behavior, or whether an independently implemented client reproduces the same vectors.

6.2 Prototype Performance

The performance profile exercises the native Ristretto255 2-of-3 TPASS path inside the five-party authenticated same-host deployment. Ten blocks use a seed-derived order over successful recovery, fixed wrong-input rejection, and recovery with party 1 unavailable. Each fresh scenario project performs one unmeasured warm-up and three measured operations, yielding 30 samples per scenario. We retain client-observed phase and total latency, application-body bytes by role, the canonical cloud-object size, and aggregate persistent bytes; we do not remove outliers.

Table **??** reports latency for the retained 30-file corpus. Each interval is computed deterministically: quartiles and percentiles use linear Type 7 interpolation, and the median confidence interval uses 10,000 deterministic bootstrap resamples. Median enrollment was 250.030 ms; median correct, one-party-unavailable, and wrong-input recovery latencies were 442.063, 411.134, and 409.090 ms. These values characterize only this exact local profile, not production practicality, scalability, concurrency, network-area performance, or independent administration.

Table 3: Same-host native 2-of-3 profile latency. CI is the deterministic bootstrap 95% interval for the median; $n = 30$ per operation.

Operation	n	Median ms	P25–P75	P05–P95	Median 95% CI
Enrollment	30	250.030	240.135–279.385	209.182–296.540	242.750–264.852
Correct recovery	30	442.063	401.352–627.816	364.473–686.572	410.514–558.880
Party 1 unavailable	30	411.134	363.720–635.975	317.735–737.163	370.996–516.955
Wrong input	30	409.090	380.480–618.600	350.906–677.782	391.633–570.320

6.3 Cloud-Only Snapshot Boundary

The versioned cloud-only snapshot scenario captures the exact canonical S3-compatible backup bytes and a public locator/integrity manifest into a fresh two-file volume. Candidate testing then runs in a separate non-root container with no credentials or network and with only that volume mounted read-only. The runner validates the object, manifest, digest, size, locator, public TPASS shape, and absence of prohibited fields before exercising two fixed synthetic candidates. Counted guards make attempted filesystem or socket access a failed observation, while a test-only injected verifier confirms that the harness detects a candidate-signal positive control.

The retained run reported two candidates, zero candidate signals, zero network attempts, zero excluded-path accesses, a valid snapshot, and no prohibited-material finding; output scanning and resource cleanup passed. Its aggregate-only record binds the clean source commit, dependency locks, pseudonymous host, configuration, interval, and exact immutable output path. This is bounded implementation evidence for the registered path, not a cryptographic proof or real-provider forensic result.

6.4 One-Party Persistent-Snapshot Boundary

The versioned one-party snapshot scenario models the below-threshold compromise in the deployed 2-of-3 profile using pre-generated synthetic state rather than a compromise mechanism. After one normal recovery, party 1 is stopped and a trusted networkless collector copies every regular file in its persistent volume into a canonical manifest-bound snapshot. This deliberately includes that party's native TPASS state, signer and TLS keys, authorization bindings, runtime package, attempt state, and SQLite companion files when present; it excludes the cloud object, client/resolver state, all other party volumes, credentials for other roles, and online service access. A separate non-root process receives only that snapshot read-only, with no credentials or network, validates its exact cryptographic/configuration/database bindings, and exercises two fixed synthetic candidates under counted file and socket guards.

The retained run reported one compromised party at threshold two, two candidates, zero candidate signals, zero network attempts, zero excluded-path accesses, zero secret-output exposures, a valid snapshot, and no cloud material; output scanning and complete resource cleanup passed. Positive controls fail the report when a test-only Boolean verifier or boundary access is introduced. This is bounded persistent-state implementation evidence for one synthetic party/profile/epoch/checkpoint, not a cryptographic proof, live-party or side-channel result, or cumulative-compromise analysis.

6.5 Combined Cloud-Plus-Party Snapshot Boundary

The versioned combined-snapshot scenario tests the actual union rather than inferring combined security from separate storage layouts. After one synthetic recovery, trusted collectors create the frozen cloud and stopped-party sub-snapshots. A credential-free, networkless finalizer independently validates both and publishes a canonical top-level manifest only if their backup identifier, epoch, backup digest, and TPASS public parameters match. A separate non-root process receives only this completed union read-only, with no credentials or network, and exercises two fixed synthetic candidates under the same counted file and socket guards. Positive controls reject a test-only verifier, boundary access, malformed or extra input, sub-manifest substitution, and a manifest-consistent union assembled from different enrollments.

The retained run reported one compromised party at threshold two, two candidates, zero candidate signals, zero network attempts, zero excluded-path accesses, zero secret-output exposures, valid cloud and party sub-snapshots, and a matched combined binding. Output scanning passed, and exact-project-label queries found no remaining containers, volumes, or networks. This is bounded implementation evidence for one synthetic persistent union, not a cryptographic proof, live-compromise result, cryptanalytic review, or cumulative-compromise analysis.

6.6 Online Guessing Model

TPASS prevents offline guessing below threshold, but it does not make user-selected structured recovery inputs unpredictable. The residual risk is online guessing: an adversary submits candidate recovery inputs through threshold recovery sessions and learns only whether a session ultimately succeeds. The relevant quantity is the conditional min-entropy h of the complete cue-derived password input given the adversary's auxiliary information A, including public traces, personal knowledge, and any resolver-observation leakage.

For an online attempt budget k, the best single guess succeeds with probability at most 2^{-h}, and a conservative union bound gives

$$\Pr[\text{success in } k \text{ attempts}] \le \min(1, k\,2^{-h}).$$

This bound is not a measurement of cue strength. It connects a deployment-enforced online budget to residual risk, while the cue policy and adversary knowledge determine h. The current prototype does not justify substituting its configured local budget for an adversarially enforced k; rate limiting and abuse prevention are deployment assumptions outside the scoped LOCUS result. The security benefit established by the underlying boundary is narrower:

cloud-only compromise, below-threshold recovery-party compromise, or their combination gives the adversary no cheaper local test than online threshold sessions. Obvious or exposed cues can still make h small, so deployments should treat cue refresh, recovery re-enrollment, and audit review as part of the operational recovery lifecycle.

7 Security Analysis

7.1 Assumptions and Claim Scope

The security of LOCUS depends on the following assumptions.

A1: TPASS security. The underlying TPASS protocol prevents coalitions of fewer than t recovery parties from reconstructing the protected secret or testing password guesses offline.

A2: AEAD and key-derivation security. The AEAD scheme provides confidentiality and integrity for sk_U, and HKDF is used with domain-separated inputs.

A3: Authenticated confidential setup. The setup channel from the user to each recovery party is authenticated and confidential.

A4: Any numerical online-attempt bound is external and conditional. Claim **??** applies only when a deployment independently enforces at most k online evaluations. The scoped LOCUS prototype does not establish that premise; its configured local counter must not be substituted for k.

A5: Cue privacy is not absolute. Human-selected cues may be guessed. Resolver providers may observe lookup queries. LOCUS therefore does not rely on cue secrecy alone.

A6: Operational boundary. Compromise of at least t recovery parties, endpoint malware that observes sk_U or p_M, and denial of service by unavailable parties are outside the confidentiality guarantees. Section **??** describes the corresponding recovery-state management requirements.

Under these assumptions, LOCUS provides the following main-body security claims. They correspond to the attacker goals summarized in Table **??**.

7.2 No-Offline-Oracle Claims

Claim 1 (Correctness). If the user enters a candidate recovery input M' that the enrolled cue policy maps to the same canonical bytes as M, at least t recovery parties participate with valid states, and the stored cloud object is the enrolled B_U, then recovery returns sk_U.

Proof sketch. The cue policy is deterministic, so $Z_{M'} = Z_M$ implies $p'_M = p_M$ by Eq. (**??**). TPASS correctness then gives $S'_R = S_R$ for a valid threshold set. The client derives the same K_{wrap} as in enrollment, and AEAD decryption under the enrolled associated data returns the original sk_U.

For the remaining claims, an *offline cue-testing oracle* is a local predicate that an adversary can evaluate on a guessed recovery input M^*, without further online recovery-party participation, and that indicates whether M^* yields the enrolled cue-derived password or a decrypting key.

Claim 2 (Cloud-only confidentiality and no offline oracle). A cloud-only adversary that obtains B_U cannot decrypt C_U or test guessed recovery inputs offline, except with negligible probability under the AEAD, key-derivation, and TPASS assumptions.

A cloud adversary obtains

$$B_U = (\mathsf{v}_B, \mathsf{bid}, e, v_R, C_U, \mathsf{pp}_{\mathsf{YHCLZK}}, \Pi_{\mathsf{ctx}}, \Pi_{\mathsf{sec}}, h_B).$$

This object contains the encrypted private key and public policy metadata, but not the TPASS server states, S_R, K_{wrap}, raw cues, selected resolver-record identifiers, Z_M, p_M, or a verifier for p_M.

Proof sketch. The only secret-bearing value in B_U is C_U, which is protected by AEAD under K_{wrap}. Deriving K_{wrap} requires S_R, and recovering S_R requires successful TPASS recovery with the enrolled password and a threshold set of parties. Since B_U contains no local verifier for p_M, no raw cues, and no recovery-party states, a guessed M^* gives the cloud adversary no local check beyond attempting online threshold recovery. The retained snapshot experiment in Section **??** checks this exact implemented storage surface and candidate path, but does not replace the cryptographic assumptions in this argument.

Claim 3 (Below-threshold party compromise). A coalition of fewer than t recovery parties cannot reconstruct S_R, recover sk_U, or test guessed recovery inputs offline from their party records.

Proof sketch. The compromised TPASS-capable party record contains one native server state and public recovery metadata. It contains no encrypted private key, raw cues, selected resolver-record identifiers, Z_M, p_M, S_R, K_{wrap}, or password verifier. Under TPASS threshold security, fewer than t server states cannot reconstruct the protected secret or validate candidate passwords offline [**? ?**]. Because the encrypted private key is stored in the cloud rather than in party records, the coalition also lacks C_U unless it separately compromises cloud storage. The retained snapshot experiment in Section **??** checks the complete stopped persistent surface of one synthetic party in the deployed 2-of-3 profile; it does not replace the TPASS assumptions or generalize to live, cumulative, or threshold compromise.

Claim 4 (Combined cloud and below-threshold compromise). An adversary that compromises the cloud and fewer than t recovery parties still does not obtain an offline cue-testing oracle.

Proof sketch. The cloud object gives the adversary C_U, v_R, context-policy metadata, and TPASS public parameters. The compromised parties give fewer than t TPASS states. A guessed recovery input M^* lets the adversary compute a candidate p^*_M, but below-threshold TPASS security prevents using p^*_M and the partial states to check the guess locally. The AEAD tag on C_U also cannot be tested until the adversary derives K_{wrap}, which requires S_R, which in turn requires successful threshold recovery. The retained experiment in Section **??** checks the exact implemented union for one matching cloud object and stopped party snapshot; it neither replaces the inherited cryptographic assumptions nor generalizes to live, cumulative, or threshold compromise.

7.3 Tamper Detection and Error Channels

Claim 5 (Tamper detection). Substitution, rollback, or corruption of the cloud backup object is detected during recovery when the

client has access to honest party metadata for the intended backup. Deletion remains an availability failure.

Proof sketch. Each party configuration stores the domain-separated digest h_B of the complete backup excluding its self-digest, binding party metadata to the enrolled cloud object. During recovery, the client validates the cloud envelope and compares its exact (bid, e, h_B) reference to party-pinned metadata before decryption. A substituted or rolled-back object with different encoded contents yields a digest mismatch, except with negligible hash-collision probability. Tampering with C_U is also detected by AEAD authentication failure. If the cloud deletes all usable copies, there may be no object to verify; LOCUS treats this as availability loss rather than confidentiality failure.

Protocol failures must not create a cheaper cue-testing channel than online recovery. The relevant failures are malformed public parameters, malformed party state, backup digest mismatch, party lockout, TPASS recovery failure, TPASS digest-check failure, and AEAD authentication failure. In LOCUS, malformed metadata and digest mismatch are checked before decryption and reveal only that the recovery material is inconsistent with the selected backup. TPASS failure or digest-check failure indicates that the online threshold protocol did not yield the enrolled group secret. AEAD failure indicates that the client did not derive the correct wrapping key or that the ciphertext was corrupted.

These failures are useful for legitimate recovery diagnostics and artifact tests, but they must be exposed carefully. Recovery parties should learn only whether they will participate, whether local policy permits the session, and what they need for their audit log. They need not learn whether the final client-side TPASS digest check or AEAD decryption succeeded. Externally visible recovery failures should be normalized to a generic rejection, with detailed causes retained only in local client diagnostics or administrative audit channels. This preserves the intended boundary: an attacker can still use the recovery interface as an online oracle, but cannot convert distinct error paths into an offline or below-threshold oracle.

7.4 Online Guessing and Attempt Controls

Claim 6 (Online guessing bound). If a deployment permits at most k counted online recovery attempts for a backup and the complete cue-derived password input has conditional min-entropy h given the adversary's auxiliary information, then the adversary's online success probability is bounded by $\min(1, k2^{-h})$, up to negligible cryptographic failure terms.

Proof sketch. By definition of conditional min-entropy, the best candidate recovery input succeeds with probability at most 2^{-h} before the adversary observes online accept/reject outcomes. Each counted recovery session evaluates at most one candidate through the threshold protocol. A union bound over k permitted attempts gives $\min(1, k2^{-h})$. The bound is a model of residual online risk, not a measurement that any cue family has entropy h.

TPASS prevents offline guessing below threshold, but it does not eliminate or rate-limit online guessing. As shown in Table **??**, public-information and social-knowledge attackers remain relevant because they may generate strong candidate cues and submit them through the recovery interface. The prototype records signed, request-bound attempt entries before native TPASS commitments and provides exact retry and selected crash behavior. These are local implementation properties, not a global adversarial budget.

A bounded compact-profile state exploration exposes the missing property. After two honest parties certify one slot while a third honest party remains at the predecessor, restoring one participating honest database lets the restored party, the already-stale honest party, and two Byzantine parties form a stale four-party view and certify a conflicting request. A second bounded trace reauthorizes after restored pre-retirement state. Thus signatures, SQLite durability, and party-quorum reconciliation do not establish rollback-resistant global attempt control. An independently administered monotonic witness or a stronger consensus design could address this class, but would add a new trust, availability, latency, and metadata boundary; LOCUS leaves that extension to future work.

Deployments must therefore supply any required online rate limiting, admission, abuse prevention, and false-lockout recovery. Such controls can deny service and must not silently reset state during ordinary recovery. The conditional parameter k in Claim **??** refers to a bound actually enforced by that deployment, not to the prototype's configured counter.

7.5 Resolver Boundary and Authority Separation

Claim 7 (Resolver-boundary leakage). For a cue policy that uses an external resolver, local canonicalization and hashing prevent the cloud and recovery parties from storing raw cue material, but they do not hide lookup activity from the resolver before local hashing.

Proof sketch. When the active policy resolves cues externally, its queries occur before the client constructs Z_M. A third-party resolver may therefore observe queries, selected candidates, timing, and account metadata. These observations can reduce the adversary's uncertainty about M, but resolver observation alone does not provide C_U, TPASS states, S_R, K_{wrap}, or a verifier for p_M. Resolver leakage therefore affects the online-guessing distribution rather than creating the offline storage oracle ruled out by Claims **??**–**??**.

LOCUS separates storage privacy from resolver privacy whenever a policy relies on external lookup. Hashing a resolved cue record before TPASS setup prevents the cloud and recovery parties from receiving raw cues, but it does not hide the lookup from a third-party resolver that processes the query before hashing. External lookup providers or directory services may therefore observe sensitive recovery-cue candidates during enrollment or recovery.

The strongest mitigation is a local-first resolver. A client can resolve cue records from encrypted local stores, user-maintained labels, cached identifiers, or deployment-managed directories without contacting a public provider during recovery. This mode reduces third-party leakage but requires users or deployments to maintain stable local records and refresh them when identifiers drift.

Where local-only resolution is impractical, deployments can reduce leakage by using self-hosted or organization-hosted directories, querying through a privacy-preserving lookup service, or batching and padding resolver queries so that a single recovery cue is less distinguishable. These mitigations have different costs: self-hosting shifts trust to the operator, private lookup can add latency and deployment complexity, and batching or padding reduces but

does not eliminate metadata leakage. A no-provider mode is also possible: the user selects stable local labels at enrollment, stores only the versioned cue policy and encrypted backup externally, and later re-enters those local labels without contacting a public resolver. This improves resolver privacy at the cost of weaker drift handling and potentially more user-entry errors.

The baseline LOCUS design therefore makes a conservative claim. Resolver providers are outside the storage-recovery privacy boundary: they can learn lookup activity unless the deployment chooses a local, self-hosted, cached, private-lookup, batched, or no-provider resolver mode. This leakage can help an attacker prioritize guesses, but it does not by itself create the offline storage oracle analyzed in Section **??**.

Claim 8 (Separation of authority). No single baseline component can recover sk_U alone under the stated threat model.

Proof sketch. Successful recovery requires three conditions:

(1) access to the encrypted cloud backup object B_U;
(2) correct regeneration of the cue-derived password p_M;
(3) online participation from at least t valid recovery parties.

The cloud has the ciphertext but not the recovery states. A recovery party has one recovery state but not the password or ciphertext. A resolver may observe cue lookups but does not receive TPASS states or the cloud ciphertext as part of the protocol. A below-threshold coalition lacks enough TPASS states. Thus no single component satisfies all three conditions, while a compromise of at least t recovery parties is outside the threshold assumption.

8 Lifecycle and Recovery State Management

LOCUS is a recovery workflow, not only a one-time cryptographic construction. Its security boundary depends on how deployments refresh state after recovery, compromise, resolver drift, and lockout events.

8.1 Successful Recovery and Re-Enrollment

A successful recovery should be treated as the start of a new enrollment epoch. The recovered client should generate a fresh recovery nonce v_R, recovery identity, TPASS secret/state, cloud object, and party records under an explicit successor epoch. If the lost device may have been compromised rather than merely unavailable, the user should also rotate the protected private key or move assets and credentials to a new key. The old B_U and R_i records should then be retired so that stale parties cannot continue participating in recovery sessions for an obsolete backup.

8.2 Party Replacement and Party Compromise

Replacing a recovery party requires re-enrollment or an equivalent authenticated re-sharing procedure that produces fresh party records and updated metadata for the active backup. A suspected below-threshold party compromise remains within the no-offline-oracle model before refresh, but it should still trigger party retirement and threshold reset because the adversary may keep partial state indefinitely and may later compromise additional parties. A compromise of at least t parties defeats the threshold assumption; operational response must assume the recovery secret may be exposed and proceed with key rotation and full re-enrollment.

8.3 Cue Refresh and Resolver Drift

Cue exposure, weak or obvious cues, changed resolver behavior, ambiguous search results, and provider identifier drift should trigger cue-policy refresh. Refresh means selecting a new recovery input or resolver mode, recording a new cue-policy version, sampling fresh public recovery metadata, and distributing fresh TPASS state. Local-first or cached resolvers reduce third-party leakage, but they shift responsibility to the user or deployment to keep local records stable enough for later recovery.

8.4 Rollback, Deletion, and Attempt-State Recovery

Cloud-object rollback and substitution are detected by comparing the downloaded B_U with current honest party-stored digest h_B, and ciphertext corruption is detected by AEAD failure. This does not detect rollback of the authoritative party metadata itself. Deletion remains an availability risk, so deployments should replicate cloud backups and maintain enough responsive parties to meet the threshold. Restored or inconsistent party state is outside the demonstrated security boundary and should be handled conservatively by refusing or requiring out-of-band re-enrollment rather than silently resetting counters or reactivating an epoch.

8.5 Artifact Coverage

The reference artifact exercises several lifecycle-relevant behaviors: wrong recovery input, unsupported policy version, insufficient parties, malformed party state, backup digest mismatch, stale cloud-object rejection, exceeded local attempt limits, AEAD failure, cross-epoch mixing, and state-separation checks. These tests support only their exact failure-handling and state-separation claims; they are not party-state rollback resistance, a complete distributed deployment, a key-rotation service, or production incident response.

9 Limitations

LOCUS does not make human-selected cues uniformly random. Obvious personal associations may be easy for a public-information or social-knowledge adversary to guess. The architecture reduces the damage of storage compromise by avoiding offline verification below threshold, but it does not itself bound the resulting online sessions; rate limiting is a deployment assumption.

For policies that use external resolution, LOCUS does not hide cue lookup activity from the resolver. A provider may observe search queries and selected candidates before local hashing occurs. Deployments that need stronger resolver privacy should use local records, self-hosted directories, private-information-retrieval techniques, or other resolver designs beyond the baseline described here.

Resolver identifiers can drift, APIs can change, and search results can become ambiguous over time. A practical deployment therefore needs cue refresh, policy versioning, and recovery guidance when a user's intended selection is unchanged but resolver output changes. Availability is also conditional: fewer than t responsive recovery parties can block legitimate recovery, and a threshold compromise defeats the threshold assumption. Party-database snapshots can

fork attempt or lifecycle state; public recovery admission, false-lockout administration, general party replacement, and independently administered deployment are absent. Finally, the research prototype supports protocol validation and selected same-host failure tests, but it is not production cryptography, independently audited, or a complete distributed service.

10 Related Work

Threshold and password-protected recovery. Shamir secret sharing established the standard (t, n) reconstruction model [**?**]; verifiable and proactive variants add checks and long-lived maintenance [**? ? ?**]. Plain threshold sharing, however, makes a threshold coalition sufficient for recovery. Password-protected secret sharing and key retrieval require an additional human-supplied secret while preventing specified server views from becoming offline password oracles [**? ? ? ? ? ? ?**]. LOCUS instantiates Yi et al.'s zero-knowledge TPASS construction; it does not claim a new TPASS protocol. Its system-level delta is the versioned cue-policy boundary, separation of encrypted cloud backup from TPASS states, and bounded state-surface evaluation.

Closest encrypted-backup systems. SafetyPin protects encrypted mobile backups under a short PIN, distributes trust across an HSM fleet, and couples recovery to hardware-enforced guessing defenses [**?**]. SVR3 distributes secret recovery across heterogeneous enclaves at different cloud providers and includes rollback-protection and fault-tolerance mechanisms; it is evaluated at deployed scale [**?**]. The WhatsApp backup protocol and subsequent PPKR work likewise study password-bootstrapped retrieval with explicit server/HSM trust assumptions [**? ?**]. These systems preclude a broad claim that LOCUS is the first distributed, offline-guessing-resistant, or attempt-controlled recovery design. They also provide stronger completed attempt-control or deployment evidence. LOCUS instead studies a hardware-independent TPASS composition, a deployment-defined cue-policy boundary, and storage separation across a cloud object and software recovery parties. Its partial quorum ledger is not comparable to their rollback protections.

Social and account recovery. Wallet and decentralized-application recovery often makes guardians, peers, or smart contracts part of the authorization factor [**? ? ?**]. LOCUS parties do not exercise social judgment: a threshold participates in TPASS, while the client must still supply the enrolled input. This avoids making helpers direct recovery authorities, but leaves public admission, abuse prevention, and false-lockout administration to a deployment. Password managers, passkey sync, and platform accounts solve related availability problems under different provider and account-recovery trust assumptions; LOCUS does not claim to replace those ecosystems.

Human-derived cues and recovery policies. Usable-security studies provide scheme-specific evidence for structured cues. Al-Ameen et al. found that verbal cues and meaningful user interaction raised one-week recognition success for system-assigned graphical passwords in a 56-participant study [**?**]. A 66-day GeoPass field study reported 96.1% successful sessions for a single geographic password, while separate multiple-password studies found substantial interference without a meaningful account–location association [**?**]. Hang et al. evaluated location-based questions as fallback authentication and reported 95% accuracy after four weeks and 92% after six months [**?**]. Zhou et al. studied a single Google Maps point with tolerant-distance matching and reported high reproduction through two weeks [**?**].

These results motivate structured cues as an interface class, but their matching tolerances, rehearsal patterns, and authentication tasks do not establish the memorability of an arbitrary policy or of the LOCUS reference policy. Human-selected passwords and personal-knowledge answers can also be biased, predictable, or exposed through public and social information [**? ?**]. LOCUS contributes the protocol boundary after cue selection: a versioned client policy produces canonical TPASS input without exporting raw cues or a verifier. The evaluated location–person policy is one instantiation of that boundary.

Fuzzy and biometric recovery. Graphical authentication studies the broader recall–guessability tension [**?**], while fuzzy commitments and extractors reconstruct keys from noisy inputs [**? ?**]. LOCUS does not perform fuzzy matching: the active cue policy must resolve candidate input to the enrolled canonical bytes or fail. Canonicalization is a protocol reproducibility property, not evidence of memorability, entropy, or long-term resolver stability.

The supplementary comparison in Table **??** summarizes these distinctions mechanism by mechanism. The comparison narrows the novelty claim. LOCUS contributes neither TPASS nor the first distributed encrypted-backup or rollback-controlled recovery system. Its scoped contribution is the applied composition and evidence boundary: an explicit client-local cue-policy contract, a native TPASS layer, ciphertext/state role separation, bounded persistent-snapshot tests, same-host cost measurements, and an explicit negative result for the attempted quorum ledger.

11 Conclusion

LOCUS treats the transformation from human-derived recovery information to cryptographic input as an explicit protocol boundary. A versioned deployment policy resolves and canonicalizes selected input locally, binds the resulting bytes to a recovery identity, and supplies the TPASS password without making that password the backup-encryption key. TPASS instead recovers an independently sampled group secret from which the client derives the wrapping key. Separating the encrypted cloud backup from recovery-party state prevents cloud compromise, below-threshold party compromise, or their combination from enabling offline cue testing under the stated assumptions. The three location–person-pair construction demonstrates one policy contract rather than defining the architecture. Policy-specific human usability, global rate limiting, party-state rollback resistance, public admission, and independent administration remain outside the scoped result.

A Open Science

The intended release artifact is designed to support reproduction of the system claims without exposing sensitive human data. The current repository includes the hybrid Rust/Python prototype, cue-policy conformance vectors, synthetic examples, 33 aggregate-only attack/performance records, a deterministic raw-to-processed pipeline, manifest-bound generated table rows, and documentation for selected state-separation, attempt-control, lifecycle, and failure-handling paths. The current corpus is a same-host collection

Table 4: Mechanism comparison with the closest recovery lines.

System/line	Human input and trust split	Attempt/rollback mechanism	Relation to LOCUS
Yi et al. TPASS [**?**]	Password plus threshold server states	Primitive does not provide deployment admission or durable global counting	Cryptographic building block instantiated by LOCUS; not a LOCUS novelty.
PPKR/WhatsApp [**? ?**]	Password-bootstrapped server key retrieval; explicit server/HSM assumptions	HSM and authenticated service assumptions depend on the construction	Closest password-to-key-retrieval line; LOCUS distributes TPASS state and separates the ciphertext role.
SafetyPin [**?**]	Short PIN; trust split across an HSM fleet	Hardware protections and a distributed recovery design constrain guessing	Stronger completed hardware-backed attempt-control and scale story; different cue and software assumptions.
SVR3 [**?**]	PIN/secret recovery across heterogeneous enclaves and cloud providers	Explicit rollback protection, usage control, and fault tolerance	Stronger deployed rollback/scale evidence; LOCUS does not claim equivalence.
Social recovery	Guardians or contracts authorize or contribute recovery	Policy is social/organizational and scheme-specific	LOCUS parties do not judge identity; exact client input remains required.
LOCUS	Versioned cue-policy input; three-pair reference policy; 2-of-3 TPASS state separated from cloud ciphertext	Durable local signed ledger, but bounded rollback counterexamples defeat a global claim	Same-host composition and snapshot evidence; no independent administration or global bound.

and still requires anonymous clean-host reproduction. The artifact also documents the research-grade boundary of the native TPASS implementation: it has not received an independent cryptographic audit.

Human-subjects records, raw cues, cue-derived identifiers tied to people, invitation or recovery emails, social-profile identifiers, API keys, and account metadata are excluded. The paper reports no original human-subjects results. If an anonymous artifact link is used during review, it will remain accessible throughout the ASIACCS review process.

B Ethical Considerations

LOCUS may involve sensitive recovery inputs because a deployment's cue policy can draw on places, relationships, local records, images, events, or other personal information, and a resolver can expose associated lookup behavior. This submission does not report original human-subjects data. Future studies of policy-specific memorability, user comprehension, guessing resistance, or comparisons with retained password strings require prior institutional approval or exemption, data-minimization procedures, retention limits, withdrawal procedures, and a release plan that excludes identifying records.

The main ethical and privacy risks are deployment risks: resolver providers may observe lookup activity, strict attempt controls can deny service to legitimate users, and weak cues can be guessed online through whatever attempts a deployment permits. The paper therefore separates storage privacy from resolver privacy and treats audit, lockout reset, cue refresh, and re-enrollment as operational requirements rather than optional usability details.

C TPASS Algebra and Instantiation Notes

This appendix records the Yi–Hao–Chen–Liu zero-knowledge-proof-based TPASS algebra instantiated by LOCUS. For readability, equations retain the source paper's multiplicative notation; the Rust core maps group multiplication/division to Ristretto point addition/subtraction and exponentiation to scalar multiplication. Every implementation transcript uses the fixed domain `LOCUS-TPASS-YI-ZK-RISTRETTO255-v1`, an operation label, and length-prefixed fields. The main paper abstracts this interface in Section **??**.

C.1 Setup Mapping

LOCUS instantiates TPASS with the zero-knowledge-proof-based t-out-of-n construction of Yi et al. [**? ?**]. g_1 maps to the canonical Ristretto basepoint; g_2 is transparently derived by SHA-512-to-Ristretto under the fixed protocol domain, replacing the source generator ceremony. The reference epoch-1 recovery identity is

$$ID_R = \texttt{"LOCUS-compose-recovery-v1:"} \,\|\, \text{bid}.$$

The password input is $p_M = H_q(\texttt{"password"}, ID_R, Z_M)$ under the transcript convention above. The protected TPASS secret exponent is z_R, and the corresponding group secret is

$$S_R = g_2^{z_R}.$$

The digest exponent is

$$\theta_R = H_q(\texttt{"secret-digest"} \| ID_R \| \texttt{Encode}(S_R)).$$

This is the LOCUS domain-separated version of Yi et al.'s $H(S)$ digest check. The recovery identity binds the reference backup identifier.

The setup phase samples three random degree-$(t-1)$ polynomials over $\mathbb{Z}_q$:

$$f_1(x), f_2(x), f_3(x),$$

with constants

$$f_1(0) = p_M, \qquad f_2(0) = z_R, \qquad f_3(0) = \theta_R.$$

For each recovery party P_i, the client computes

$$p_i = f_1(i), \qquad z_i = f_2(i), \qquad \theta_i = f_3(i).$$

The server state stored by P_i is

$$\text{st}_i = (i, p_i, z_i, \theta_i).$$

A coalition of fewer than t parties obtains fewer than t evaluations of each polynomial and therefore cannot reconstruct the constants, following Shamir secret sharing [**?**].

C.2 Recovery Equations

Recovery uses exactly t parties. If more than t are available, the client selects a threshold subset

$$I \subseteq \{1, \dots, n\}, \qquad |I| = t.$$

For each $i \in I$, define the Lagrange coefficient at zero:

$$a_i^{(I)} = \prod_{\substack{j \in I \\ j \neq i}} \frac{j}{j - i} \pmod q. \tag{5}$$

Then

$$\sum_{i \in I} a_i^{(I)} p_i = p_M, \qquad \sum_{i \in I} a_i^{(I)} z_i = z_R, \qquad \sum_{i \in I} a_i^{(I)} \theta_i = \theta_R.$$

During recovery, the fresh device derives a candidate password p'_M, samples

$$r \leftarrow \mathbb{Z}_q^*,$$

and computes the TPASS retrieval request

$$A = g_1^r g_2^{-p'_M}. \tag{6}$$

The negative exponent follows the Yi et al. sign convention and ensures cancellation when $p'_M = p_M$.

Each party P_i, $i \in I$, samples

$$\rho_i, \alpha_i, \beta_i \leftarrow \mathbb{Z}_q^*$$

and computes

$$B_i = g_1^{\rho_i} g_2^{a_i^{(I)} p_i}, \tag{7}$$

$$C_i = g_1^{\alpha_i}, \tag{8}$$

$$D_i = g_1^{\beta_i}. \tag{9}$$

It then forms the transcript

$$\tau_i = ID_R \| \mathsf{Encode}(I) \| \mathsf{Encode}(i) \| \mathsf{Encode}(A) \| \mathsf{Encode}(B_i) \| \mathsf{Encode}(C_i) \| \mathsf{Encode}(D_i),$$

and computes

$$h_i = H_q(\texttt{"server-proof-challenge"} \| \tau_i), \tag{10}$$

$$H_i = H_q(\texttt{"server-proof-second-challenge"} \| \mathsf{Encode}(h_i)), \tag{11}$$

$$\delta_i = h_i \alpha_i + H_i \beta_i \pmod q. \tag{12}$$

The tuple (C_i, D_i, δ_i) is the non-interactive proof-of-knowledge component used by the Yi et al. zero-knowledge-proof variant. Each party makes

$$\langle ID_R, B_i, C_i, D_i, \delta_i \rangle$$

available within the selected party set. Each selected party recomputes h_j, H_j for the other broadcasts and verifies

$$g_1^{\delta_j} = C_j^{h_j} D_j^{H_j} \qquad \text{for each received } j \in I. \tag{13}$$

Invalid proofs cause the session to abort.

After all proofs are accepted, each party computes

$$C = \prod_{j \in I} C_j, \tag{14}$$

$$D = \prod_{j \in I} D_j, \tag{15}$$

and forms

$$\tau = ID_R \| \mathsf{Encode}(I) \| \mathsf{Encode}(A) \| \mathsf{Encode}(C) \| \mathsf{Encode}(D),$$

$$h = H_q(\texttt{"aggregate-challenge"} \| \tau), \tag{16}$$

$$W = A \prod_{j \in I} B_j. \tag{17}$$

Then party P_i computes

$$E_i = g_2^{a_i^{(I)} z_i h} C^{-\rho_i} W^{\alpha_i}, \tag{18}$$

$$F_i = g_2^{a_i^{(I)} \theta_i h} D^{-\rho_i} W^{\beta_i}. \tag{19}$$

The party sends

$$\langle ID_R, C, D, E_i, F_i \rangle$$

to the client.

The client combines responses as

$$E = \prod_{i \in I} E_i, \tag{20}$$

$$F = \prod_{i \in I} F_i. \tag{21}$$

It computes

$$S'_R = (E / C^r)^{h^{-1}}, \tag{22}$$

$$T'_R = (F / D^r)^{h^{-1}}, \tag{23}$$

where h^{-1} is the inverse of h modulo q. The client accepts the TPASS output only if

$$T'_R = g_2^{H_q(\texttt{"secret-digest"} \| ID_R \| \mathsf{Encode}(S'_R))}. \tag{24}$$

C.3 Correctness Sketch

If $p'_M = p_M$, then by Eqs. (**??**) and (**??**),

$$A \prod_{i \in I} B_i = g_1^r g_2^{-p_M} \prod_{i \in I} g_1^{\rho_i} g_2^{a_i^{(I)} p_i}.$$

Using

$$\sum_{i \in I} a_i^{(I)} p_i = p_M,$$

the password-dependent terms cancel:

$$A \prod_{i \in I} B_i = g_1^{r + \sum_{i \in I} \rho_i}.$$

The remaining Yi et al. recovery equations then yield

$$S'_R = S_R \qquad \text{and} \qquad T'_R = g_2^{\theta_R}.$$

Therefore Eq. (**??**) holds. If $p'_M \neq p_M$, the cancellation does not occur and the final digest relation fails except with negligible probability under the assumptions of the TPASS construction.

The Rust tests validate these equations, canonical message encodings, malformed inputs, all small threshold subsets, and one deterministic cross-language vector. This is regression evidence from one algebraic implementation, not an independent cryptographic audit. The source proof also assumes the proof-of-knowledge property of its server proof; implementing the verification equation does not establish that assumption independently.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.